

Git and GitHub Workflow Guide

A hands-on guide for learning professional Git collaboration using your current test automation project and two systems.

Push and pull

Branches

Pull requests

Code review

Merge conflicts

Forks

System A

your local copy

push
pull

GitHub

shared cloud repo

push
pull

System B

second local copy

Git tracks local history. GitHub shares that history with other machines and people.

Private repo note: your current migration snapshot includes `.env` and `playwright/.auth`. Keep this GitHub repository private. In a company workflow, these files are normally never committed.

What you will practice

- Move work from System A to GitHub to System B.
- Create feature branches instead of editing `main` directly.
- Open, review, update, and merge pull requests.
- Resolve conflicts when two people edit the same lines.

How to use this PDF

- Use tiny changes first, mainly in `README.md` or `docs/git-practice.md`.
- Run each exercise exactly once, then repeat it without looking.
- Focus on purpose first, command syntax second.

Core Concepts You Must Know

Repository

The project plus its Git history. Your local repo lives on your laptop. The remote repo lives on GitHub.

Commit

A saved snapshot with a message. Think of it as a named checkpoint.

Branch

A separate line of work. Use branches to isolate changes before merging into `main`.

Remote

A named GitHub location. Most repos use `origin` for the default remote.

Pull Request

A GitHub discussion around merging one branch into another. This is where review happens.

Merge

Combines branch history. After review, the feature branch is merged into `main`.

Branch picture



The feature branch starts from `main`, gets its own commits, then returns through a pull request.

Check state

```
git status
```

Save snapshot

```
git add README.md
git commit -m "Update README"
```

Share snapshot

```
git push
```

Your First Two-System Workflow

1. System B clones once

```
git clone https://github.com/venu-ratcha/mentorcloud-qa.git
cd mentorcloud-qa
```

2. System A changes and pushes

```
git status
git add README.md
git commit -m "Update README from system A"
git push
```

3. System B receives changes

```
git checkout main
git pull
```

clone

Download the repository for the first time. You usually clone only once per machine.

push

Send your local commits to GitHub.

pull

Bring GitHub commits into your current local branch.

status

Your safest command. Use it before and after almost every Git action.

Exercise 1: round trip

1. On System A, add one line to `README.md`.
2. Commit and push.
3. On System B, run `git pull` and confirm the line appears.
4. Reverse the direction: edit on System B, push, then pull on System A.

Feature Branch and Pull Request Workflow

Start from latest `main`

```
git checkout main
git pull
```

Create branch

```
git checkout -b
add-login-test-
note
```

Commit work

```
git add README.md
git commit -m
"Add login test
note"
```

Push branch

```
git push -u
origin add-login-
test-note
```

Open PR on GitHub
review, discuss,
merge

Why companies use branches

- `main` stays stable and usable.
- Work can be reviewed before it affects others.
- Tests can run automatically before merge.
- Multiple developers can work independently.

Good branch names

- `fix/login-timeout-test`
- `add/session-report-checks`
- `docs/git-practice-notes`
- `refactor/auth-fixtures`

Rule: for real collaboration, do not push directly to `main`. Create a branch, open a PR, let another person review, then merge.

Pull Request Review Like a Company

PR author

1. Explain what changed.
2. Keep the PR small.
3. Include test evidence when relevant.
4. Respond to comments with fixes or reasoning.

Reviewer

1. Check correctness first.
2. Check readability and maintainability.
3. Ask for changes when needed.
4. Approve only when comfortable merging.



Pull request: Add login test note

Checks passed - Playwright smoke tests

Files changed: README.md

Reviewer comment: "Can you mention System B too?"

Approved - ready to merge

After merge, update both systems

```
git checkout main
git pull
```

```
git branch -d add-login-test-note
git push origin --delete add-login-test-note
```

How Merge Conflicts Happen and How to Fix Them

System A

Changes the same README line to:

Run tests from System A.

Pushes and merges first.

System B

Changes that same README line to:

Run tests from System B.

Pulls later and gets a conflict.

same line

```
<<<<<< HEAD
Run tests from System B.
=====
Run tests from System A.
>>>>>> main
```

1 Open the file with conflict markers.

2 Choose the correct final text.

3 Delete the markers.

4 Stage and commit the fix.

```
git add README.md
git commit -m "Resolve README conflict"
git push
```

Pull, Fetch, Merge, Rebase, Fork

Command or concept	Use it when	Simple meaning
<code>git fetch</code>	You want to see remote updates without changing your files.	Download information only.
<code>git pull</code>	You are ready to update your current branch from GitHub.	Fetch plus merge.
<code>git merge main</code>	You want to bring <code>main</code> into your branch and preserve history.	Combine histories.
<code>git rebase main</code>	Your branch is private and you want a cleaner history before PR.	Replay your commits on top.
Branch	You have write access to the same repository.	Separate work inside one repo.
Fork	You do not have write access, or you want your own copy.	Your personal copy of another repo.

Do I need to update my local branch?

Just inspect remote changes
git fetch

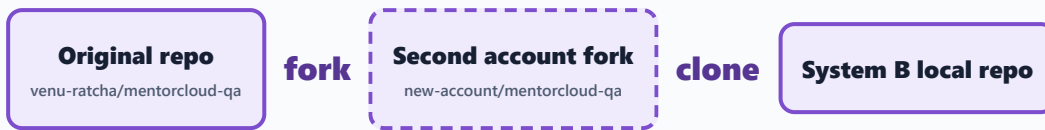
Bring changes into my files
git pull

Do I need to contribute?

I have repo access
create branch

I do not have repo access
fork first

Using a Second GitHub Account



Set remotes

```
git remote -v
git remote add upstream https://github.com/venu-
ratcha/mentorcloud-qa.git
git fetch upstream
```

Sync fork with original

```
git checkout main
git fetch upstream
git merge upstream/main
git push origin main
```

Fork workflow: push your branch to your fork, then open a PR into the original repository. This is common in open-source projects and external vendor work.

Real-World Developer Routine

1

Pull latest main

2

Create feature branch

3

Make small commits

4

Push branch

5

Open PR

6

CI tests run

7

Review and fix

8

Merge

Daily start

```
git checkout main
git pull
git checkout -b feature/my-task
```

Daily finish

```
git status
git add .
git commit -m "Meaningful message"
git push
```

Professional rules

- Keep PRs small enough to review in 10 to 20 minutes.
- Commit messages should explain the change, not just say "update".
- Pull latest `main` before starting new work.
- Never commit secrets in normal projects.
- Resolve comments with a new commit on the same branch.

Four-Week Hands-On Plan

Week 1: basics

- Clone on System B.
- Push from A, pull on B.
- Push from B, pull on A.

Week 2: branches and PRs

- Create branches for small README changes.
- Open PRs from both systems.
- Review and merge from the other system.

Week 3: conflicts

- Edit the same line from both systems.
- Resolve the conflict manually.
- Repeat until conflict markers are familiar.

Week 4: fork workflow

- Create a second GitHub account.
- Fork the repo.
- Open a PR from fork to original.

Commands to memorize

```
git status
git branch
git checkout main
git checkout -b feature-name
git add .
git commit -m "Clear message"
git push -u origin feature-name
git pull
git fetch
git log --oneline --graph --decorate --all
git diff
```

Start with safe documentation edits. Once the workflow feels natural, apply it to real Playwright test changes.